# Documentation

Major Functions and how they work:
DTSC-3601-Final-Project/INITALDEMOFILESHERE/.ipynb_checkpoints

**CountsExtraction_Demo-checkpoint.ipynb**

**fetch_annual_traffic_counts():**
Fetches annual traffic counts from the NCDOT API for specified location IDs.

Args: None, the IDs are pre-specified in the function.

Returns: ncdot_df: A DataFrame containing traffic data (NCDOT average annual traffic counts) for the specified locations.

**save_traffic_counts_to_s3(ncdot_df):**
Saves the traffic counts DataFrame to an S3 bucket in CSV format with the current timestamp.
Args: ncdot_df: A DataFrame containing traffic data (NCDOT average annual traffic counts).
Returns: None, but prints the saved file with the associated timestamp

**SpeedsExtraction_Demo-checkpoint.ipynb**

**fetch_tomtom_traffic_data():**
Fetches traffic data from the TomTom API for specified latitude and longitude coordinates and returns a DataFrame containing the relevant information.

Args: None; the latitude and longitude coordinates are hardcoded in the function.

Returns: tomtom_df (DataFrame): A DataFrame containing the traffic data for the specified coordinates, including timestamp, latitude, longitude, current speed, free flow speed, current delay, and confidence

**edit_time_features(tomtom_df):**
Adds time features to the TomTom traffic DataFrame; including hour, day of the week, month, and speed ratio as properties of existing features (timestamp and current_speed vs. free_flow_speed).

Args: tomtom_df (DataFrame): A DataFrame containing the traffic data.

Returns: tomtom_df (DataFrame): The DataFrame with the added features. speed_ratio is calculated as current_speed divided by free_flow_speed. The time features are extracted from the timestamp and include hour, day_of_week, and month

**add_holiday_info(tomtom_df):**
Adds holiday information to the TomTom traffic DataFrame by mapping timestamps to US federal holidays.

Args: tomtom_df (DataFrame): A DataFrame containing the traffic data.

Returns: tomtom_df (DataFrame): The DataFrame with an added 'holiday' column indicating whether each timestamp falls on a US federal holiday or not

**save_tomtom_data_to_s3(tomtom_df):**
Saves the traffic information DataFrame to an S3 bucket in CSV format with the current timestamp.

Args:  tomtom_df: A DataFrame containing traffic data (NCDOT average annual traffic counts).

Returns: None

**Inital_Demo-checkpoint.ipynb**

**extract_tomtom_traffic_data():**
Extracts traffic data from the specified S3 bucket and prefix, decompresses the files if necessary, and combines them into a single DataFrame.

Args: None

Returns: tomtom_df: pd.DataFrame: A DataFrame containing the combined traffic data from all relevant files in the S3 bucket.

**school_holiday_check(traffic_df):**
Checks if a given date falls within the defined school holiday periods (defined in the function for UNCC).

Args: traffic_df (pd.DataFrame): A DataFrame containing traffic data with a 'timestamp' column to be compared to school holiday periods.

Returns: traffic_df (pd.DataFrame): The input DataFrame with an additional 'school_holiday' column indicating whether each date falls within a school holiday period. 1 for holiday, 0 for non-holiday. For days of the week, Monday = 0, Sunday = 6.

**extract_ncdot_traffic_data_s3():**
Extracts traffic data from the specified S3 bucket and key for NCDOT traffic data, decompresses the file if necessary, and returns it as a DataFrame with coordinates edited to match the other data (account for slight decimal differences).

Args: None

Returns: ncdot_df: pd.DataFrame: A DataFrame containing the NCDOT traffic data.

**def extract_ncdot_traffic_data(ncdot_df):**
For a provided DataFrame of NCDOT traffic data, edits the coordinates to match the TomTom data perfectly (account for slight decimal differences). Pared down to only the coordinate matching part of the S3 extraction function, so it can be used on any DataFrame of NCDOT traffic data.

Args: ncdot_df (pd.DataFrame): A DataFrame containing the NCDOT traffic data.

Returns: ncdot_df (pd.DataFrame): The input DataFrame with updated coordinates to match the TomTom data.

**combine_dataframes(tomtom_df, ncdot_df):** Combines two DataFrames (tomtom_df and ncdot_df) by matching latitude and longitude values, and adds the AADT_2022 column from ncdot_df to tomtom_df.

Args:
tomtom_df (pd.DataFrame): The first DataFrame containing traffic data with latitude and longitude columns.
ncdot_df (pd.DataFrame): The second DataFrame containing traffic data with matched_latitude, matched_longitude, and AADT_2022 columns.

Returns: traffic_df (pd.DataFrame): A combined DataFrame with the AADT_2022 column added from ncdot_df to tomtom_df.

**preprocess_train_data(combined_df):**

Preprocesses the combined DataFrame by cleaning the data, creating new features, and preparing it for machine learning. Trains an XGBoost regression model on the preprocessed data.

Args: combined_df (pd.DataFrame): The combined DataFrame containing all the data.

Returns: model (xgb.XGBRegressor): The trained XGBoost regression model.

**evaluate_model(model, X_test, y_test):**
Evaluates the performance of the trained model on the test set and prints various performance metrics.

Args:
model: xgb.XGBRegressor: The trained XGBoost regression model.
X_test (pd.DataFrame): The test set features.
y_test (pd.Series): The actual target values for the test set.
Returns: None; it displays the evaluation metrics and predicted outputs for some inputs in the test set.

**predict_custom_traffic(model, location_id, timestamp_str):**
Takes a single location_id and timestamp string, automatically assigns the hardcoded free_flow_speed and AADT_2022, runs feature engineering, and predicts traffic speed and congestion level.

Args:
model: xgb.XGBRegressor: The trained XGBoost regression model.
location_id (int): The location ID for which to predict traffic speed.
timestamp_str (str): The timestamp string in the format 'YYYY-MM-DD HH:MM:SS' for which to predict traffic speed.
Returns: tuple: A tuple containing the predicted traffic speed (float) and congestion level (str).